



PYTHON

FOR AI & ML

Complete Python Foundation — From Syntax to Neural Networks
Your first step into the world of Artificial Intelligence.

PART 01 — PYTHON FOUNDATIONS

- 01 📁 Variables, Numbers & Strings
- 02 📄 Lists, Dictionaries, Sets & Tuples
- 03 📄 If Conditions & For Loops
- 04 🔧 Functions & Lambda Functions
- 05 📁 Modules & pip install
- 06 📁 Read & Write Files
- 07 🛡️ Exception Handling
- 08 🏠 Classes & Objects

PART 02 — ADVANCED PYTHON

- 09 🔗 Inheritance
- 10 ⚙️ Generators
- 11 🔁 Iterators
- 12 📄 List Comprehensions
- 12 📄 List Comprehensions
- 13 🎀 Decorators
- 14 🍹 Multithreading
- 15 ⚡ Multiprocessing

• BEGINNER → ADVANCED • PYTHON 3.x • AI / ML READY •

1. Variables, Numbers & Strings

What is a Variable?

Think of a variable like a labeled box. You put something inside the box, give it a name, and later you can open the box and use what's inside!

Variables

Variables store information. You create them by writing: name = value

```
# Creating variables (like labeling boxes)
name = "Alex"           # a box called "name" holding "Alex"
age = 10                # a box called "age" holding 10
is_happy = True         # a box holding True or False

# Printing them out
print(name)             # Output: Alex
print(age)              # Output: 10
print(is_happy)         # Output: True
```

Numbers

Python has two kinds of numbers: whole numbers (int) and decimal numbers (float).

```
# Integers (whole numbers)
cookies = 5
players = 11

# Floats (decimal numbers)
price = 3.99
temperature = 36.6

# Math with numbers
print(5 + 3)    # 8 (addition)
print(10 - 4)   # 6 (subtraction)
print(3 * 4)    # 12 (multiplication)
print(15 / 4)   # 3.75 (division)
print(15 // 4)  # 3 (floor division - no decimals!)
print(2 ** 3)   # 8 (2 to the power of 3)
```

Strings

Strings are text — anything inside quotes. You can play with them in cool ways!

```
# Creating strings
greeting = "Hello, World!"
```

```
city = 'New Delhi'

# Joining strings (called concatenation)
first = "Harry"
last = "Potter"
full_name = first + " " + last
print(full_name)          # Harry Potter

# f-strings (the easiest way to mix text and variables!)
age = 10
print(f"I am {age} years old")  # I am 10 years old

# Useful string tricks
word = "python"
print(word.upper())           # PYTHON
print(word.capitalize())     # Python
print(len(word))              # 6   (how many letters?)
print(word[0])                # p   (first letter)
```

Remember This!

Variables are just names for boxes of data.

Use " " or ' ' around text to make strings.

Use f-strings like f"Hello {name}" to mix variables into text!

2. Lists, Dictionaries, Sets & Tuples

💡 Why do we need these?

Instead of making 100 separate variables, we can store many things in one place using these special containers!

📋 Lists — Like a Shopping List

A list holds many items in order. Items can be changed.

```
# Creating a list
fruits = ["apple", "banana", "mango", "orange"]

# Getting items (counting starts from 0!)
print(fruits[0])      # apple  (1st item)
print(fruits[2])      # mango  (3rd item)
print(fruits[-1])     # orange (last item)

# Adding & removing
fruits.append("grape") # add to end
fruits.remove("banana") # remove banana

# Check how many items
print(len(fruits))    # 4

# Slicing (getting part of a list)
print(fruits[1:3])    # ["banana", "mango"]
```

📖 Dictionaries — Like a Contact Book

A dictionary stores data as key:value pairs. Like a real dictionary — you look up a word (key) and get its meaning (value)!

```
# Creating a dictionary
student = {
    "name": "Arjun",
    "age": 12,
    "grade": "7th"
}

# Getting values
print(student["name"]) # Arjun
print(student["age"])  # 12

# Adding / changing values
student["city"] = "Mumbai" # add new key
student["age"] = 13         # update existing

# All keys and values
```

```
print(student.keys())    # dict_keys(["name", "age", "grade", "city"])
print(student.values())  # dict_values(["Arjun", 13, "7th", "Mumbai"])
```

Sets — Only Unique Items

Sets are like bags that NEVER hold duplicate items. Useful for removing duplicates!

```
# Creating a set
colors = {"red", "blue", "green", "red", "blue"}
print(colors)          # {"red", "blue", "green"} - duplicates gone!

# Adding and removing
colors.add("yellow")
colors.discard("red")

# Sets are great for checking membership
print("blue" in colors) # True
```

Tuples — A Locked List

Tuples are like lists, but they CANNOT be changed once created. Good for fixed data.

```
# Creating a tuple (use round brackets)
point = (10, 20)
dimensions = (1920, 1080)

# Accessing items (same as lists)
print(point[0])      # 10
print(point[1])      # 20

# You CANNOT change a tuple
# point[0] = 5        # This would cause an ERROR!
```

Container	Key Feature
 List []	Ordered, changeable, allows duplicates
 Dictionary { key:val }	Key-value pairs, fast lookup
 Set { }	No duplicates, unordered
 Tuple ()	Ordered, cannot be changed

3. If Condition & For Loop

☒ If Condition — Making Decisions

If conditions let your program make decisions — just like you do in real life!

🌸 Real Life Example

IF it is raining → take umbrella

ELIF it is sunny → wear sunglasses

ELSE → just go outside normally

```
# Basic if-elif-else
marks = 85

if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")      # This will print!
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: F")

# Comparison operators
# == means 'is equal to'
# != means 'is NOT equal to'
# >  means 'greater than'
# <  means 'less than'
# >= means 'greater than or equal to'
# <= means 'less than or equal to'

# Combining conditions
age = 15
has_ticket = True

if age >= 13 and has_ticket:
    print("You can enter the movie!")
```

🔄 For Loop — Doing Things Again & Again

A for loop repeats an action for each item in a list (or a range of numbers).

```
# Loop through a list
pets = ["dog", "cat", "fish", "rabbit"]

for pet in pets:
    print(f"I love my {pet}!")
# Output:
# I love my dog!
```

```
# I love my cat!
# I love my fish!
# I love my rabbit!

# Loop with range() - count from 1 to 5
for i in range(1, 6):
    print(f"Count: {i}")

# Loop with range() counting by 2s
for i in range(0, 10, 2):
    print(i)                # 0 2 4 6 8

# Useful: enumerate gives index AND value
fruits = ["apple", "mango", "banana"]
for i, fruit in enumerate(fruits):
    print(f"{i+1}. {fruit}")
# 1. apple
# 2. mango
# 3. banana
```

break & continue

break → "STOP the loop right now!"

continue → "Skip this one and go to the next"

```
# break example
for i in range(10):
    if i == 5:
        break                # stop when i reaches 5
    print(i)                 # prints 0 1 2 3 4

# continue example
for i in range(6):
    if i == 3:
        continue            # skip 3
    print(i)                 # prints 0 1 2 4 5
```

4. Functions & Lambda Functions

What is a Function?

A function is like a recipe card. You write the instructions once, and then you can use them as many times as you want!

Functions

```
# Define a function using 'def'
def greet(name):
    message = f"Hello, {name}! Welcome to Python!"
    return message

# Call the function
result = greet("Priya")
print(result)    # Hello, Priya! Welcome to Python!

# Function with multiple parameters
def add(a, b):
    return a + b

print(add(3, 5))    # 8
print(add(10, 20)) # 30

# Function with default values
def power(base, exponent=2):    # default exponent is 2
    return base ** exponent

print(power(4))        # 16 (4 squared)
print(power(2, 10))    # 1024 (2 to the 10th)
```

Lambda Functions — Mini Functions

Lambda is a tiny one-line function. Great for simple, quick tasks!

```
# Normal function
def square(x):
    return x * x

# Same thing as a lambda!
square = lambda x: x * x
print(square(5))    # 25

# Lambda with two inputs
add = lambda a, b: a + b
print(add(3, 7))    # 10

# Super useful with sorted()
students = [("Alice", 95), ("Bob", 87), ("Charlie", 92)]
```

```
sorted_students = sorted(students, key=lambda s: s[1])
print(sorted_students)
# [('Bob', 87), ('Charlie', 92), ('Alice', 95)]
```

When to use which?

Use `def` when your function has multiple lines or you'll reuse it many times.

Use `lambda` for quick, one-line operations — especially with `sorted()` or `map()`.

5. Modules (pip install)

What is a Module?

Modules are like toolboxes made by other programmers.
Instead of building everything from scratch, you borrow their tools!
pip install = downloading a new toolbox from the internet.

Built-in Modules (already included!)

```
# math module - for mathematical operations
import math

print(math.sqrt(25))    # 5.0 (square root)
print(math.pi)         # 3.141592...
print(math.ceil(4.2))  # 5   (round up)
print(math.floor(4.8)) # 4   (round down)

# random module - for random numbers
import random

print(random.randint(1, 6)) # random number between 1 and 6 (like a dice!)
print(random.choice(["rock", "paper", "scissors"])) # random pick

# datetime module - for dates and times
import datetime

today = datetime.date.today()
print(today)                # 2025-01-01 (today's date)
```

Installing External Modules with pip

pip is Python's package manager. Use it in your terminal/command prompt:

```
# In your terminal (not in Python code):
pip install requests      # for making web requests
pip install numpy        # for math with big datasets
pip install pandas       # for working with data tables

# -----
# Using the 'requests' module (after installing)
import requests

response = requests.get('https://api.github.com')
print(response.status_code) # 200 means success!

# Using 'from' to import only what you need
from math import sqrt, pi
print(sqrt(16))            # 4.0   (no need to write math.sqrt!)
print(pi)                 # 3.14159...
```

Most Popular AI/Data Science Modules

numpy → fast math with numbers and arrays

pandas → work with data like Excel tables

matplotlib → draw graphs and charts

scikit-learn → machine learning!

tensorflow / pytorch → deep learning and AI

6. Reading & Writing Files

💡 Why Read/Write Files?

Programs often need to save data (like saving your game progress!) or read data from a file (like loading a list of names).

📖 Reading Files

```
# Reading an entire file
file = open('story.txt', 'r')    # 'r' means read
content = file.read()
print(content)
file.close()                    # always close the file!

# Better way: using 'with' (auto-closes the file!)
with open('story.txt', 'r') as file:
    content = file.read()
    print(content)

# Reading line by line
with open('names.txt', 'r') as file:
    for line in file:
        print(line.strip())      # .strip() removes \n at end

# Reading all lines into a list
with open('names.txt', 'r') as file:
    lines = file.readlines()
print(lines)    # ['Alice\n', 'Bob\n', 'Charlie\n']
```

✍️ Writing Files

```
# Writing to a file ('w' creates new or overwrites!)
with open('diary.txt', 'w') as file:
    file.write("Day 1: I learned Python today!\n")
    file.write("Day 2: I wrote my first program!\n")

# Appending to a file ('a' adds to end, doesn't overwrite)
with open('diary.txt', 'a') as file:
    file.write("Day 3: I built a calculator!\n")

# Writing multiple lines at once
lines = ["Apple\n", "Banana\n", "Mango\n"]
with open('fruits.txt', 'w') as file:
    file.writelines(lines)
```

Mode	What it does
"r" (read)	Open file for reading. Error if file doesn't exist
"w" (write)	Create new file, or OVERWRITE if exists

"a" (append)	Add to end of file, create if doesn't exist
"r+" (read+write)	Open for both reading and writing

7. Exception Handling

💡 What is an Exception?

An exception = an ERROR that crashes your program.

Exception handling = catching that error so your program doesn't crash!

It's like a safety net for your code. 🕸

🛡 try / except / finally

```
# Without exception handling - this CRASHES!
number = int(input("Enter a number: "))
print(10 / number)    # CRASH if number = 0, or text entered!

# With exception handling - safe!
try:
    number = int(input('Enter a number: '))
    result = 10 / number
    print(f"Result: {result}")

except ValueError:
    print("Oops! Please enter a NUMBER, not text!")

except ZeroDivisionError:
    print("Oops! You cannot divide by zero!")

except Exception as e:
    print(f"Something went wrong: {e}")    # catch anything else

finally:
    print("This always runs, no matter what!")
```

🔗 Common Exceptions

<code>ValueError</code>	Wrong type of value (text where number expected)
<code>ZeroDivisionError</code>	Dividing by zero
<code>FileNotFoundError</code>	Trying to open a file that doesn't exist
<code>IndexError</code>	Accessing list item that doesn't exist
<code>KeyError</code>	Accessing dictionary key that doesn't exist
<code>TypeError</code>	Wrong type (like adding number + text)

```
# Real world example: safely reading a file
try:
    with open('data.txt', 'r') as f:
        content = f.read()
    print("File loaded successfully!")
```

```
except FileNotFoundError:
    print("Sorry, the file does not exist!")

# Raising your own error
def check_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative!")
    return f"Age is valid: {age}"
```

8. Classes & Objects

Classes vs Objects — The Cookie Cutter Analogy

A CLASS is like a cookie cutter — it's the template/blueprint.

An OBJECT is the actual cookie made from that cutter.

You can make MANY cookies from ONE cookie cutter!

Creating a Class

```
# Define a class (the blueprint)
class Dog:

    # __init__ runs automatically when you create a dog
    def __init__(self, name, breed, age):
        self.name = name        # store the name
        self.breed = breed      # store the breed
        self.age = age          # store the age

    # Methods = functions inside a class
    def bark(self):
        print(f"{self.name} says: Woof! Woof!")

    def info(self):
        print(f"{self.name} is a {self.breed}, {self.age} years old.")

# Creating objects (making actual dogs from the blueprint!)
dog1 = Dog("Bruno", "Labrador", 3)
dog2 = Dog("Mochi", "Shih Tzu", 1)

# Using the objects
dog1.bark()      # Bruno says: Woof! Woof!
dog2.info()      # Mochi is a Shih Tzu, 1 years old.
print(dog1.name) # Bruno
```

Inheritance — Child Classes

Inheritance means a new class can BORROW everything from another class and add more!

```
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating.")

# Child class inherits from Animal
class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name) # call parent's __init__
```

```
        self.color = color

    def meow(self):
        print(f"{self.name} says: Meow!")

# Cat has BOTH eat() and meow()
kitty = Cat("Whiskers", "orange")
kitty.eat()      # Whiskers is eating. (from Animal!)
kitty.meow()     # Whiskers says: Meow!
```

Key OOP Concepts

Class → blueprint/template

Object → real thing made from blueprint

`__init__` → runs when object is created

`self` → refers to the current object

Method → a function inside a class

Inheritance → child class borrows from parent class



Quick Reference Cheat Sheet

```
# — VARIABLES —————
name = "Alex"    age = 10    pi = 3.14    is_fun = True

# — STRINGS —————
f"Hello {name}"    word.upper()    len(word)    word[0]

# — LISTS —————
fruits = ["a","b","c"]    fruits.append("d")    fruits[0]

# — DICTIONARY —————
d = {"key": "value"}    d["key"]    d["new"] = "new_val"

# — IF CONDITION —————
if x > 5:    elif x == 5:    else:

# — FOR LOOP —————
for item in list:    for i in range(10):

# — FUNCTION —————
def greet(name):    return f'Hi {name}'

# — LAMBDA —————
square = lambda x: x**2

# — IMPORT —————
import math    from random import randint

# — FILES —————
with open('f.txt','r') as f: content = f.read()

# — EXCEPTIONS —————
try:    ...    except ValueError:    ...    finally: ...

# — CLASS —————
class Dog:    def __init__(self,name): self.name=name
```



You're a Python Superstar!



Keep coding, keep learning, and have fun! 

PART 02

ADVANCED PYTHON

Inheritance · Generators · Iterators · Comprehensions · Decorators · Threading

1. 🍷 Inheritance

🍷 What is Inheritance?

Imagine a pizza shop. Every pizza (Margherita, BBQ Chicken, Veggie) has the same base: dough + sauce + cheese. That's the PARENT class. Each specific pizza ADDS its own toppings. That's the CHILD class! Child classes inherit everything from the parent and add their own stuff.

Single Inheritance — One Parent

```
# Parent class
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def speak(self):
        print(f"{self.name} says {self.sound}!")

    def sleep(self):
        print(f"{self.name} is sleeping... 🐾")

# Child class — inherits from Animal
class Dog(Animal):
    def __init__(self, name):
        super().__init__(name, "Woof") # call parent __init__

    def fetch(self):
        # extra dog-only method
        print(f"{self.name} fetches the ball! 🏀")

class Cat(Animal):
    def __init__(self, name):
        super().__init__(name, "Meow")

    def purr(self):
        print(f"{self.name} purrrrs... 🐱")

# Using them
dog = Dog("Bruno")
cat = Cat("Whiskers")

dog.speak()    # Bruno says Woof!      ← inherited from Animal
dog.sleep()    # Bruno is sleeping... ← inherited from Animal
dog.fetch()    # Bruno fetches the ball! ← Dog's own method

cat.speak()    # Whiskers says Meow!
cat.purr()     # Whiskers purrrrs...
```

Multilevel Inheritance — Grandparent → Parent → Child

```

class Vehicle:                # Grandparent
    def move(self):
        print("Moving...")

class Car(Vehicle):           # Parent
    def honk(self):
        print("Beep beep! 🚗")

class ElectricCar(Car):       # Child
    def charge(self):
        print("Charging battery... ⚡")

tesla = ElectricCar()
tesla.move()                 # Moving...    ← from Vehicle (grandparent!)
tesla.honk()                 # Beep beep!   ← from Car (parent)
tesla.charge()              # Charging...   ← ElectricCar's own

```

Method Overriding — Replacing Parent's Method

```

class Shape:
    def area(self):
        print("I don't know my area yet!")

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):                # OVERRIDE parent's area()
        print(f"Area = {3.14 * self.radius**2:.2f}")

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):                # OVERRIDE again
        print(f"Area = {self.w * self.h}")

Circle(5).area()               # Area = 78.50
Rectangle(4, 6).area()        # Area = 24

```

Key Points

`super()` → calls the parent class's method
 Child classes get ALL methods & attributes from the parent
 Override → rewrite a parent method in the child class
`isinstance(dog, Animal)` → True (checks if object is from a class)

2. Generators

What is a Generator?

Imagine an ice cream machine. Instead of making ALL 1000 scoops upfront (which would overflow!), it makes ONE scoop at a time — only when you ask. A Generator works the same way: it produces values ONE at a time, saving huge amounts of memory! 🧠

yield — The Magic Word

```
# Normal function - returns ALL at once
def get_numbers_normal(n):
    result = []
    for i in range(n):
        result.append(i * i)    # store everything in memory
    return result

# Generator - yields ONE at a time
def get_squares(n):
    for i in range(n):
        yield i * i            # pause here, give value, resume later

# Using the generator
gen = get_squares(5)

print(next(gen))    # 0    (asks for next value)
print(next(gen))    # 1
print(next(gen))    # 4
print(next(gen))    # 9
print(next(gen))    # 16

# Or loop through all
for square in get_squares(6):
    print(square)    # 0, 1, 4, 9, 16, 25
```

Real World Example — Reading a Huge File

```
# BAD: loads entire file into memory (crashes with huge files!)
def read_all_lines(filename):
    with open(filename) as f:
        return f.readlines()    # ALL lines at once = memory problem!

# GOOD: generator reads ONE line at a time
def read_lines_lazy(filename):
    with open(filename) as f:
        for line in f:
            yield line.strip()    # one line at a time

# Use it
for line in read_lines_lazy("bigfile.txt"):
```

```
print(line)
```

Generator Expression — One-line Generator

```
# List comprehension — makes everything NOW (uses memory)
squares_list = [x**2 for x in range(1000000)] # 1 million items in RAM!

# Generator expression — makes items on demand (saves memory)
squares_gen = (x**2 for x in range(1000000)) # almost no memory used!

# Both work the same way to iterate
print(next(squares_gen)) # 0
print(next(squares_gen)) # 1
print(sum(squares_gen)) # add up all remaining squares
```

Normal Function	Generator
Returns ALL values at once	Yields ONE value at a time
Uses lots of memory	Uses very little memory
Uses return keyword	Uses yield keyword
Runs to completion	Pauses at each yield

3. Iterators

What is an Iterator?

An iterator is like a bookmark in a book.

It remembers WHERE you are and lets you read the NEXT page.

Any object you can loop through with 'for' is an iterable.

An iterator is an iterable that remembers its position.

__iter__ and __next__ — The Magic Methods

```
# Every iterator must have these two methods:
# __iter__() → returns itself
# __next__() → returns the next value (or raises StopIteration)

class CountUp:
    def __init__(self, start, stop):
        self.current = start
        self.stop = stop

    def __iter__(self):
        return self          # I am my own iterator

    def __next__(self):
        if self.current > self.stop:
            raise StopIteration # signal: we're done!
        value = self.current
        self.current += 1
        return value

# Use it in a for loop
for num in CountUp(1, 5):
    print(num)      # 1  2  3  4  5

# Or manually
counter = CountUp(10, 12)
print(next(counter)) # 10
print(next(counter)) # 11
print(next(counter)) # 12
# print(next(counter)) # StopIteration error!
```

iter() and next() with built-in types

```
# Lists, strings, etc. are ITERABLES (not iterators by default)
fruits = ["apple", "banana", "mango"]

# Convert to iterator using iter()
fruit_iter = iter(fruits)

print(next(fruit_iter)) # apple
print(next(fruit_iter)) # banana
```

```
print(next(fruit_iter))    # mango
# print(next(fruit_iter)) # StopIteration!

# Strings are also iterable!
letter_iter = iter("Hello")
print(next(letter_iter))   # H
print(next(letter_iter))   # e
```

Iterators vs Iterables

Iterable	Iterator
Can be looped with for	Can be looped AND tracks position
Examples: list, string, dict	Examples: iter(list), generator
Does NOT have __next__	HAS both __iter__ and __next__
Can create many iterators	One-way — can't go back!

Simple Rule

ALL iterators are iterables, but NOT all iterables are iterators.

Think of it like: all dogs are animals, but not all animals are dogs! 🐶

4. List Comprehensions

❏ What is a List Comprehension?

It's a magical shortcut to create lists in ONE line!

Instead of writing a for loop + append, you write it all in [].

It's faster to write AND faster to run. Python developers LOVE it!

Basic Syntax

```
# Formula: [expression for item in iterable]

# The OLD way (4 lines)
squares = []
for x in range(1, 6):
    squares.append(x ** 2)
print(squares)    # [1, 4, 9, 16, 25]

# The COMPREHENSION way (1 line!)
squares = [x ** 2 for x in range(1, 6)]
print(squares)    # [1, 4, 9, 16, 25]

# More examples
evens     = [x for x in range(20) if x % 2 == 0]
print(evens)    # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

words     = ["hello", "world", "python"]
upper     = [w.upper() for w in words]
print(upper)    # ['HELLO', 'WORLD', 'PYTHON']
```

With if Condition — Filtering

```
# [expression for item in iterable if condition]

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Only even numbers
evens = [n for n in numbers if n % 2 == 0]
print(evens)    # [2, 4, 6, 8, 10]

# Only numbers greater than 5
big    = [n for n in numbers if n > 5]
print(big)      # [6, 7, 8, 9, 10]

# Filter a list of strings
names  = ["Alice", "Bob", "Charlie", "Anna", "Dave"]
a_names = [n for n in names if n.startswith("A")]
print(a_names)  # ['Alice', 'Anna']
```

Dictionary & Set Comprehensions

```
# Dictionary comprehension {key: value for item in iterable}
words = ['cat', 'elephant', 'dog', 'hippopotamus']
lengths = {word: len(word) for word in words}
print(lengths) # {'cat': 3, 'elephant': 8, 'dog': 3, ...}

# Set comprehension - curly braces, no duplicates
numbers = [1, 2, 2, 3, 3, 3, 4]
unique_sq = {n**2 for n in numbers}
print(unique_sq) # {1, 4, 9, 16}

# Nested list comprehension (matrix!)
matrix = [[row * col for col in range(1, 4)] for row in range(1, 4)]
print(matrix)
# [[1, 2, 3],
#  [2, 4, 6],
#  [3, 6, 9]]
```

When NOT to use comprehensions

If the logic is complex or has many conditions — use a normal for loop.

Readability > cleverness! Code is read more often than it's written.

Rule: if it doesn't fit on one line clearly, write it the normal way.

5. 🦊 Decorators

🍰 What is a Decorator?

Imagine you have a plain cake (your function).

A decorator is like adding frosting, sprinkles & candles on top —

it wraps your function and adds extra features WITHOUT changing the cake inside!

Decorators are functions that take a function and return a better function.

Functions Inside Functions — The Foundation

```
# In Python, functions can be passed around like variables!
def greet():
    print("Hello!")

def run_twice(func):
    func()          # takes a function as argument
    func()          # call it once
    func()          # call it again

run_twice(greet)   # Hello!
                  # Hello!
```

Creating a Decorator

```
# A decorator wraps a function
def shout_decorator(func):
    def wrapper():
        print("📢 GET READY...")
        func()          # call the original function
        print("🍰 DONE!")
    return wrapper       # return the wrapped version

# Define a regular function
def say_hello():
    print("Hello, World!")

# Apply decorator the LONG way
say_hello = shout_decorator(say_hello)
say_hello()
# 📢 GET READY...
# Hello, World!
# 🍰 DONE!
```

The @ Symbol — Clean Decorator Syntax

```
# The @ symbol is a shortcut for applying decorators!

def shout_decorator(func):
```

```

def wrapper():
    print("📢 GET READY...")
    func()
    print("🏁 DONE!")
    return wrapper

@shout_decorator          # same as: say_hello = shout_decorator(say_hello)
def say_hello():
    print("Hello, World!")

say_hello()    # 📢 GET READY... / Hello, World! / 🏁 DONE!

```

Real World Decorators — Timing & Logging

```

import time

# Timer decorator — how long does a function take?
def timer(func):
    def wrapper(*args, **kwargs):      # *args lets it accept any arguments
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)                      # wait 1 second
    print("Done!")

slow_function()
# Done!
# slow_function took 1.0012 seconds

# Stacking multiple decorators
def bold(func):                        # wraps output in **bold**
    def wrapper(*a, **k):
        print("***", end="")
        func(*a, **k)
        print("***", end="")
    return wrapper

@timer
@bold
def greet(name):
    print(f"Hello {name}!", end="")

greet("Alex")    # **Hello Alex!** then timing info

```

🌟 Where are Decorators Used in Real Life?

@staticmethod, @classmethod → in class definitions

`@app.route('/home')` → in web frameworks like Flask

`@login_required` → check if user is logged in

`@timer`, `@retry` → in APIs and backend systems

6. 🧵 Multithreading

👤 What is Multithreading?

Imagine one chef in a kitchen who can stir soup, watch the oven AND chop vegetables — switching between tasks very quickly.
Threads are like multiple tasks running inside ONE program, sharing the same memory. Great for tasks that WAIT (like downloading files)!

Creating Threads

```
import threading
import time

def download_file(filename):
    print(f"⬇ Starting download: {filename}")
    time.sleep(2) # simulate download taking 2 sec
    print(f"✅ Done: {filename}")

# Without threading (one at a time - slow!)
# download_file("video.mp4") # wait 2 sec
# download_file("music.mp3") # wait another 2 sec = 4 sec total

# WITH threading (at the same time!)
t1 = threading.Thread(target=download_file, args=("video.mp4",))
t2 = threading.Thread(target=download_file, args=("music.mp3",))

t1.start() # start thread 1
t2.start() # start thread 2 (doesn't wait for t1!)

t1.join() # wait for t1 to finish
t2.join() # wait for t2 to finish

print("All downloads complete!")
# Both downloads happen at the SAME TIME → done in ~2 sec instead of 4!
```

Thread with a Class

```
import threading

class CounterThread(threading.Thread):
    def __init__(self, name, count):
        super().__init__()
        self.name = name
        self.count = count

    def run(self):
        # runs when .start() is called
        for i in range(1, self.count + 1):
            print(f"{self.name}: {i}")
```

```
t1 = CounterThread("Thread-A", 3)
t2 = CounterThread("Thread-B", 3)

t1.start()    # Thread-A and Thread-B
t2.start()    # run at the same time!
t1.join()
t2.join()
```

Thread Safety — Lock

When two threads try to change the same variable, things can go wrong. Use a Lock!

```
import threading

counter = 0
lock = threading.Lock()    # only one thread at a time

def increment():
    global counter
    for _ in range(100000):
        with lock:          # lock the door while changing counter
            counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)
t1.start(); t2.start()
t1.join(); t2.join()

print(counter)    # 200000 (correct! without lock it might be wrong)
```

Multithreading Limitation — The GIL

Python has a Global Interpreter Lock (GIL).

It means only ONE thread runs Python code at a time (even on multiple CPUs).

Threads are GREAT for I/O tasks (downloads, file reading, web requests).

For heavy CPU work (calculations), use Multiprocessing instead! 

7. ⚡ Multiprocessing



What is Multiprocessing?

Now imagine a restaurant with MULTIPLE chefs, each with their OWN kitchen!
They work completely independently — each gets their own memory.
Multiprocessing = multiple processes, each using a different CPU core.
This is how you TRULY do things in parallel and use all your CPU power!

Basic Multiprocessing

```
from multiprocessing import Process
import time

def heavy_calculation(n):
    print(f"Process {n}: starting heavy work...")
    total = sum(i**2 for i in range(10_000_000))    # 10 million calculations
    print(f"Process {n}: done! Sum = {total}")

if __name__ == '__main__':    # IMPORTANT: always need this guard!
    p1 = Process(target=heavy_calculation, args=(1,))
    p2 = Process(target=heavy_calculation, args=(2,))

    p1.start()    # runs on CPU core 1
    p2.start()    # runs on CPU core 2 — truly parallel!

    p1.join()
    p2.join()
    print("Both processes finished!")
```

Process Pool — Easiest Way

```
from multiprocessing import Pool

def square(n):
    return n * n

if __name__ == '__main__':
    numbers = [1, 2, 3, 4, 5, 6, 7, 8]

    # Pool of 4 workers (like 4 chefs)
    with Pool(processes=4) as pool:
        results = pool.map(square, numbers)    # distribute work!

    print(results)    # [1, 4, 9, 16, 25, 36, 49, 64]
```

Sharing Data Between Processes

```
from multiprocessing import Process, Value, Array
```

```
def add_hundred(shared_num):
    shared_num.value += 100

if __name__ == '__main__':
    # Shared variable between processes
    num = Value('i', 0)      # 'i' = integer, start at 0

    p = Process(target=add_hundred, args=(num,))
    p.start()
    p.join()

    print(num.value)      # 100
```

Multithreading 🧵	Multiprocessing ⚡
Shares memory (same process)	Separate memory (different process)
Limited by Python's GIL	No GIL — true parallelism
Best for I/O tasks (web, files)	Best for CPU tasks (math, AI)
Lightweight, starts fast	Heavier, more startup cost
threading module	multiprocessing module

🧠 Simple Decision Guide

Downloading files / web requests → use Multithreading 🧵
 Heavy calculations / data processing → use Multiprocessing ⚡
 Simple sequential tasks → just use regular code!



Advanced Python — Cheat Sheet

Inheritance

```
class Child(Parent):
    # inherit
    def __init__(self): super().__init__() # call parent init
    def method(self): ...                 # add / override methods
```

Generators

```
def gen(): yield value                # generator function
g = gen(); next(g)                   # get next value
gen_expr = (x**2 for x in range(10)) # generator expression
```

Iterators

```
class MyIter:
    def __iter__(self): return self
    def __next__(self): ... or raise StopIteration
```

List Comprehensions

```
[x**2 for x in range(10)]           # basic
[x for x in range(10) if x % 2 == 0] # with filter
{k: v for k, v in items}             # dict comprehension
```

Decorators

```
def decorator(func):
    def wrapper(*args, **kwargs):
        ... # before
        result = func(*args, **kwargs)
        ... # after
        return result
    return wrapper

@decorator # apply it!
def my_function(): ...
```

Multithreading

```
import threading
t = threading.Thread(target=func, args=(arg,))
t.start(); t.join()
lock = threading.Lock() # thread safety
```

Multiprocessing

```
from multiprocessing import Process, Pool
p = Process(target=func, args=(arg,))
p.start(); p.join()
with Pool(4) as pool: results = pool.map(func, items)
```

 **You've mastered Advanced Python!** 

These tools power real AI, web apps, and data systems. Keep building!  